

Fault-Tolerant and Load-Balanced IIoT Edge–Fog Architecture over Multiple Kubernetes Clusters

Benyapa Tawunwoot*, Proud Pitugkittikul†, Ramona Bhuthontharaj‡,
Nang Yein Mo Hsai§, Somchart Fugkeaw¶

School of Information, Computer, and Communication Technology (ICT)

Sirindhorn International Institute of Technology (SIIT), Thammasat University, Thailand

Email: *6622770509@g.siit.tu.ac.th, †6622770665@g.siit.tu.ac.th, ‡6622782140@g.siit.tu.ac.th,

§m6822041015@g.siit.tu.ac.th, ¶somchart@siit.tu.ac.th

Abstract—This paper proposes a fault-tolerant and load-balanced Industrial Internet of Things (IIoT) edge–fog architecture deployed over multiple Kubernetes clusters for secure, storage-efficient data processing with fault-resilient operation. The framework combines fog-scoped sensor AES keys, trusted execution environment (TEE)-enforced key transformation, Paillier slot aggregation, gossip-based load balancing, sensor-side ACK redirects, and cloud-layer ciphertext-policy attribute-based encryption (CP-ABE). To address workload dynamics and fog failures in heterogeneous fog environments, fog nodes exchange status through a decentralized gossip protocol and check the most recent workload table on every incoming sensor reading. Overload is handled through KMM-provisioned enclave delegation without sensor retransmission, while ACK timeout allows sensors to redirect encrypted readings after direct fog silence and notify the KMM for delegated key provisioning. Vacated aggregation slots are filled with Paillier encryptions of zero when the original fog node remains alive, and the key management module combines one aggregate ciphertext per fog node at each 500 ms window close. Overall, the proposed scheme jointly supports fog-scoped key isolation, adaptive per-reading delegation, bounded-loss ACK-based recovery, storage-efficient aggregation, and resilient fog operation. The 500 ms aggregation window is satisfied under hardware-accelerated assumptions during non-delegation windows; delegation and failure-recovery windows incur an additional 150 ms TEE key provisioning step, as characterized in the analytical latency model (E7).

Index Terms—Industrial Internet of Things, Edge–fog computing, Fault tolerance, Load balancing, Trusted execution environment, Paillier encryption, Kubernetes

I. INTRODUCTION

The Industrial Internet of Things (IIoT) has become a foundational pillar of Industry 4.0, enabling continuous monitoring, predictive maintenance, and automated control across large-scale industrial environments [?]. Numerous sensors and control devices generate time-sensitive data streams that impose strict requirements on latency, reliability, and security. For example, vibration sensors on rotating machinery produce hundreds of measurements per second, while safety monitoring systems in chemical plants must detect abnormal conditions in real time to prevent hazardous incidents [?].

Centralized cloud architectures struggle to satisfy these latency requirements due to the long communication path between sensors and remote data centers. Edge and fog computing have emerged as the dominant solution, moving

computation closer to data sources [?]. In an edge–fog architecture, edge gateways perform initial preprocessing near IIoT devices while fog nodes provide intermediate aggregation and analytics before forwarding results to the cloud, significantly reducing latency and bandwidth overhead.

Despite these advantages, fog-based IIoT deployments face three critical challenges. First, heterogeneous fog nodes subject to static workload assignment create resource imbalance, where weaker nodes become overloaded while stronger nodes remain underutilized [?]. Second, fault-tolerance mechanisms in fog environments introduce extra computational and storage overhead, which affects performance in resource-constrained settings [?]. Third, many existing systems require fog nodes to decrypt sensor data before performing aggregation, violating the zero-trust assumption of modern distributed infrastructures and exposing sensitive industrial data at the fog layer [?].

Prior work addresses each challenge independently but creates new tradeoffs when combined. Global shared AES key schemes allow efficient key management but mean that a single compromised fog node exposes all sensor data across the entire deployment. Per-reading Paillier encryption [?] preserves fog-layer privacy but transmits n ciphertexts per window to the cloud, causing storage overhead that grows linearly with sensor count. Replication-based fault tolerance ensures service continuity but duplicates already-large ciphertexts, amplifying the storage problem. No existing work eliminates all three tradeoffs within a single unified architecture.

Beyond the fog layer, most existing IIoT architectures deploy analytics services within a single Kubernetes cluster, which remains vulnerable to cluster-level failures that can interrupt all services simultaneously [?]. Such failures are particularly critical in industrial environments requiring continuous operation, such as predictive maintenance and power grid monitoring [?].

To address these limitations, this paper proposes a fault-tolerant and load-balanced IIoT edge–fog architecture deployed over multiple Kubernetes clusters. The proposed framework integrates fog-scoped AES key isolation enforced by Intel SGX trusted execution environments [?], Paillier homomorphic slot aggregation [?], gossip-based adaptive load balancing, and CP-ABE cloud-layer access control [?], providing end-to-end data confidentiality without exposing plaintext at

any intermediate processing stage.

The primary novelty of this work is the integration of fog-scoped key isolation with a three-layer fault-handling hierarchy for fog-based IIoT. Each fog node holds only its own scoped AES key k_{fogI} sealed inside SGX hardware, while temporary delegation is performed by the KMM provisioning the delegating node's scoped key to the receiving enclave for one aggregation window and revoking it after the window closes. The hierarchy combines: (i) fog-initiated TEE delegation when self-detected workload exceeds threshold τ , providing immediate and zero-data-loss overload handling; (ii) sensor-initiated ACK-timeout redirect using the local gossip table and CapacityScore (S6) LS-default selector with KMM provisioning in approximately 150 ms; and (iii) gossip-based neighbor detection as a fallback for simultaneous fog-node and sensor-fog channel failure.

II. RELATED WORK

This section reviews existing literature relevant to the proposed task-aware IIoT fog computing framework across three areas: (A) load balancing in fog environments, (B) fault tolerance mechanisms for distributed fog systems, and (C) privacy-preserving data processing in IIoT and fog computing. For each area, we summarise representative approaches, identify their key limitations, and explain how those limitations motivate the design choices in the proposed framework.

A. Load Balancing in Fog Computing

Load balancing in fog computing is concerned with distributing incoming workloads efficiently across available nodes to prevent bottlenecks and maximise resource utilisation [?]. The challenge is compounded by the heterogeneous capacity of fog nodes which differ in CPU, memory, and network bandwidth and by the diversity of IIoT task types, which impose widely varying computational demands. Recent surveys identify three dominant strategy families: centralised scheduling, decentralised peer-to-peer coordination, and gossip-based dissemination [?], [?].

1) *Optimisation-Based Approaches*: Early optimisation-based methods applied metaheuristics such as Particle Swarm Optimisation and Ant Colony Optimisation to allocate workflows across fog nodes [?]. The FOCALB architecture combined fitness-based node selection with fog-aware scheduling to reduce execution time and energy consumption for scientific workflow applications [?]. While FOCALB demonstrated the value of capacity-aware routing, it targets batch workflows rather than the continuous, high-frequency sensor streams characteristic of IIoT deployments, and provides no mechanism for differentiating among task types with heterogeneous computational profiles.

Multi-criteria decision-making frameworks extended optimisation approaches by jointly evaluating CPU load, network latency, and queue depth. Ebrahim and Hafid proposed an ELECTRE-based outranking method that ranks fog nodes by pairwise concordance and discordance across multiple criteria [?]. Although this approach produces theoretically

principled decisions, its $O(N^2)$ pairwise comparison complexity is prohibitive under high IIoT task arrival rates. The authors explicitly deployed the method on resource-rich SDN controllers, introducing a centralised coordinator that becomes a single point of failure. A follow-up hybrid model combining FAHP and FTOPSIS improved utilisation in fog-cloud simulations [?] but similarly assumed a centralised control plane. Ahmed et al. proposed an adaptive multi-criteria framework for fog-cloud resource allocation that achieved load balance improvements over conventional round-robin and threshold baselines [?]; however, their weight assignment relied on expert judgement rather than empirical calibration against measured sensor workloads, limiting applicability to heterogeneous IIoT environments.

2) *Reinforcement Learning Approaches*: Reinforcement learning has been applied to fog load balancing to handle dynamic workloads without static scheduling rules. Ebrahim and Hafid proposed a Double Deep Q-Learning agent that minimises waiting time while preserving node-load privacy in multi-provider deployments [?]. More recently, the same group extended this to a fully distributed Multi-Agent Reinforcement Learning (MARL) framework in which agents operate within local collaboration regions, incorporating a gossip-based observation protocol to address the impracticality of real-time global state visibility [?]. While MARL improves scalability compared to centralised RL, these approaches require an offline training phase and may need retraining when network topology or task arrival distributions change—a significant limitation in safety-critical IIoT systems where operational continuity is mandatory. Furthermore, neither the RL nor the MARL formulations differentiate among sensor types: vibration, pressure, temperature, and flow sensors are treated as a homogeneous task class despite their substantially different computational demands.

3) *Gossip-Based and Decentralised Approaches*: Gossip protocols have attracted attention for fog load management because they disseminate status information among nodes through random pairwise exchanges without requiring a central coordinator. Kashani et al. surveyed gossip and epidemic-style load balancing mechanisms, noting their low per-node message overhead and natural scalability but identifying their limited integration with task-aware delegation policies as an open problem [?]. A recent systematic literature review covering 113 peer-reviewed studies from 2020 to 2024 confirmed that gossip-based coordination remains predominantly used for monitoring rather than as a complete scheduling substrate, and highlighted the lack of heterogeneous task-type differentiation as a critical gap across the field [?].

A key limitation shared by all three strategy families is their treatment of tasks as homogeneous units, typically classified only by delay sensitivity. In practice, IIoT workloads differ substantially: a vibration sensor task involving FFT-based spectral analysis is compute-intensive ($O(n \log n)$), whereas a temperature monitoring task involves only threshold evaluation ($O(n)$). Assigning identical routing weights to these tasks leads to suboptimal delegation decisions, par-

ticularly under heterogeneous node configurations. The proposed framework addresses this gap by introducing a CapacityScore (S6) formula with two delay-class weight configurations and a task-type compatibility term—a BW-prioritized, latency-sensitive configuration for time-critical tasks and a ϕ^2 -penalised CPU-compatibility configuration for compute-heavy tasks—adapting delegation decisions to the computational and structural profile of each task class without requiring a centralised controller.

B. Fault Tolerance in Fog Computing

Fog environments are susceptible to node failures caused by hardware faults, network instability, or transient resource exhaustion. A comprehensive 2024 taxonomy of fault-tolerance approaches in distributed and cloud computing organised existing methods into reactive, proactive, adaptive, and hybrid categories [?]. In the fog context, the dominant techniques remain replication, checkpointing, and resubmission-based scheduling.

1) *Replication and Checkpointing*: Replication-based methods maintain backup copies of tasks across multiple fog nodes so that computation can continue when a primary node fails. Liu et al. proposed a replica-based scheduling strategy for heterogeneous cloud-fog environments that minimises the number of replicas needed while satisfying reliability constraints [?]. Rajab and Younis introduced a dynamic fault-tolerant scheduling scheme combining checkpointing and replication that periodically stores task states, enabling recovery without full restart when failures occur [?]. Both approaches have demonstrated reliability improvements; however, they introduce additional storage overhead and computational cost proportional to the replication factor, which is problematic for resource-constrained fog nodes. Furthermore, these methods address load balancing and fault tolerance as separate concerns, leading to fragmented system designs and increased coordination overhead.

2) *Metaheuristic and ML-Based Fault-Tolerant Scheduling*: Hybrid metaheuristic methods have been proposed to jointly optimise workload distribution and fault-tolerant scheduling. Ghasemi proposed the MOHHO algorithm combining Modified Harris Hawks Optimisation with multi-objective service placement in fog computing, which identifies critical tasks and selectively applies replication or resubmission strategies upon node failure [?]. A 2024 hybrid fault-tolerant scheduling framework using fuzzy logic-based task criticality assessment integrated with dual-backup resubmission demonstrated reduced energy overhead compared to uniform replication, evaluated under failure rates of 1-10% in iFogSim [?]. While these approaches improve reliability, they require offline parameter tuning and incur convergence overhead that may delay task re-assignment under real-time failure conditions. Mohammadi et al. studied fault tolerance specifically in fog-based Social IoT environments and highlighted that combining failure detection with load redistribution in a unified lightweight mechanism remains an open challenge [?].

3) *Gossip-Based Failure Detection*: Gossip protocols were originally proposed for distributed database maintenance and epidemic information dissemination [?]. Their application to failure detection in fog networks follows naturally from their decentralised nature: nodes declare a peer unavailable when it fails to respond for a predefined number of consecutive gossip rounds, without requiring a centralised health monitor. Naha et al. proposed a fuzzy logic-based failure handling mechanism built on gossip-style status exchange for fog nodes [?], demonstrating that decentralised detection can adapt robustly to varying failure rates. However, this work did not integrate failure detection with task-aware load balancing in a unified scheduling framework.

The proposed framework addresses the gap between fault detection and scheduling by combining gossip-based failure detection with the same CapacityScore (S6) delegation mechanism used for overload-driven load balancing. When a node is declared unavailable after missing r consecutive gossip rounds, its pending tasks are reassigned to the highest-scoring available neighbour using the most recently maintained gossip state table, requiring no additional inter-node communication beyond what is already exchanged for load monitoring.

C. Privacy-Preserving Data Processing in IIoT and Fog Computing

Protecting sensitive IIoT sensor data during transit and processing at intermediate fog nodes is a central security requirement. Prior work has explored symmetric encryption at the sensor layer, privacy-preserving aggregation at fog nodes, and fine-grained access control at the cloud boundary.

1) *Homomorphic Encryption for Fog Aggregation*: Homomorphic encryption enables computation directly on ciphertexts without decryption, making it suitable for privacy-preserving aggregation at fog nodes. Wang et al. proposed a Paillier-based privacy-preserving aggregation scheme for fog-assisted IIoT that demonstrated fault tolerance by handling missing sensor readings through blinding factor adjustments [?]. Shang et al. combined Paillier homomorphic encryption with ECDSA signatures to build a fault-tolerant data aggregation scheme that resists collusion attacks between the fog node and the cloud [?]. A recent fog-enabled secure analytics framework (FESDAO) extended Boneh-Goh-Nissim homomorphic encryption to support not only summation but statistical operations mean, variance, and ANOVA directly on encrypted data at the fog layer, without requiring decryption before analysis [?]. These works collectively confirm the feasibility of fog-layer homomorphic aggregation; however, none addresses the dynamic workload delegation scenario in which sensors are reassigned mid-aggregation-window, which is central to the proposed framework.

The MOZAIK platform demonstrated an end-to-end privacy-preserving IoT-to-cloud architecture using both Secure Multi-Party Computation and Fully Homomorphic Encryption, showing that encrypted data pipelines can satisfy practical latency requirements [?]. However, MOZAIK targets cloud-centralised analytics and does not incorporate fog-layer load

balancing or mid-window delegation semantics. The proposed framework directly addresses this gap by introducing a slot protocol that maintains aggregate correctness under dynamic sensor reassignment, using zero-fill ciphertexts to preserve IND-CPA indistinguishability of vacated slots.

2) *Hybrid Cryptographic Pipelines*: Practical IIoT systems increasingly combine symmetric, asymmetric, and homomorphic primitives into layered pipelines that balance security with performance. Qin et al. surveyed proxy re-encryption schemes and showed that PRE-based delegation achieves sub-15 ms overhead for typical IIoT payloads on commodity fog hardware [?], validating delegation latency as a practical concern. However, PRE requires key material to be present at an intermediate node during re-encryption, which creates a trust boundary conflict in hostile-fog threat models.

The proposed framework replaces the PRE delegation layer with a Trusted Execution Environment (TEE) channel implemented via Intel SGX enclaves and a Key Management Module. This eliminates the need for the intermediate node to hold re-encryption key material in host memory, bounding the blast radius of fog node compromise to a single fog-scoped key group. This design follows the direction identified by recent surveys on secure fog computing, which emphasise hardware-rooted trust as the emerging approach for overcoming the limitations of software-only delegation schemes [?], [?].

3) *Fine-Grained Access Control at the Cloud Boundary*: Ciphertext-Policy Attribute-Based Encryption (CP-ABE) enforces data-centric access control by embedding an access policy directly into the ciphertext, so that only authorised principals satisfying the policy can decrypt. Li et al. demonstrated CP-ABE-based access control for smart grid data stored in the cloud, showing that the scheme scales gracefully as the number of attribute authorities and authorised consumers grows—a property that suits multi-tenant Kubernetes analytics deployments [?]. Yang et al. integrated CP-ABE enforcement at the fog tier before data reaches the cloud, reducing the exposure window for sensitive IIoT data [?]. In the proposed framework, CP-ABE is applied at the cloud boundary to protect the aggregated ciphertext rather than at the fog layer, consistent with an honest-but-curious cloud threat model where fog nodes are trusted to aggregate but not to store plaintext results.

III. PRELIMINARIES

A. System Architecture

As shown in Fig. 1, the proposed system consists of seven layers: the IIoT sensor layer, edge gateway layer, fog SGX enclave layer, fog Paillier aggregation layer, fog load-balancing layer, cloud storage layer, and Kubernetes-based analytics services. The key management module (KMM) additionally performs a lightweight window-combine step that receives one Paillier aggregate from each fog node every $W = 500\text{ ms}$ and combines them homomorphically.

- **IIoT Sensor Layer**: IIoT sensors collect environmental or industrial monitoring data. Each sensor sends four AES-128-GCM encrypted readings per aggregation window (one every 125 ms) using the fog-scoped key of



Fig. 1. Architecture of the proposed secure IIoT fog-cloud framework.

its assigned fog node. With 100 sensors across five fog nodes (20 per node), this yields 400 readings per 500 ms window.

- **Edge Gateway Layer**: A local gateway verifies the AES-GCM authentication tag and forwards accepted ciphertexts to the assigned fog node.
- **Fog SGX Enclave Layer**: Each fog node hosts an SGX enclave that seals its local k_{fogI} , decrypts accepted AES readings, and encrypts scaled readings under Paillier without exposing plaintext to the host OS. sk_P and k_{store} are held exclusively in the dedicated storage-preparation enclave provisioned by the KMM at window close.
- **Fog Paillier Aggregation Layer**: Fog host memory maintains a running Paillier aggregate C_{agg, F_i} for the current $W = 500\text{ ms}$ window using additive homomorphic operations.
- **Fog Load-Balancing Layer**: Fog nodes exchange workload status through gossip every $T_g = 500\text{ ms}$. Delegation is checked per incoming reading using *LastKnownWorkload* and the proposed *CapacityScore* (S6) selector, then executed through KMM-provisioned enclave processing.
- **Key Management and Window Combine**: The KMM provisions keys only after remote attestation, maintains *slot_map*, and combines one aggregate ciphertext per fog node at window close.
- **Cloud Storage and Kubernetes Analytics**: The cloud stores only AES-encrypted aggregates and CP-ABE-wrapped storage keys. Authorized multi-cluster Kubernetes services recover the storage key and decrypt the aggregate for analytics.

B. Subsystem Interaction Overview

The subsystems interact in a sequential pipeline with a conditional branch at the fog layer. Under normal operation, sensor-encrypted data flows from the IIoT Sensor Layer through the Edge Gateway to the primary Fog Node, where the enclave converts AES ciphertext into Paillier ciphertext using that fog node’s scoped key. The fog host accumulates a running Paillier aggregate during the current window. When $LastKnownWorkload(F_A) \geq \tau$ for an incoming reading, Fog A selects the receiving fog node with the proposed CapacityScore (S6) rule, zero-fills that reading’s local slot, notifies the KMM, and delegates the original AES ciphertext to Fog B after the KMM provisions k_{fogA} to Enclave B. At each window close, the KMM combines one Paillier aggregate per fog node without decryption and sends the final aggregate to an enclave for storage preparation. The complete data pipeline can be summarized as

$$m_i \rightarrow C_i^{AES} \rightarrow C_i^P \rightarrow C_{agg, F_i} \rightarrow C_{agg}^{final} \rightarrow C_{data} \quad (1)$$

Sensor data is encrypted, transformed inside an attested enclave for Paillier aggregation, combined by the KMM at window close, and securely stored in the cloud.

C. Threat Model

The threat model assumes: (i) sensors are trusted but physically vulnerable to eavesdropping; each sensor holds its own copy of k_{fogI} , provisioned by the KMM at initialization, for AES-GCM encryption and for HMAC verification of gossip table summaries; (ii) the edge gateway is a trusted component provisioned with fog-scoped keys for AES-GCM tag verification only—it forwards ciphertexts without decrypting application payloads; (iii) the fog-node host operating system is honest-but-curious, meaning it correctly forwards messages and performs Paillier additions but may attempt to inspect plaintext, intermediate values, or enclave memory; (iv) the TEE enclave on each fog node is trusted after successful remote attestation; (v) the KMM is trusted for fog-scoped key provisioning, slot mapping, revocation, and homomorphic window combine but never decrypts Paillier ciphertexts; (vi) the cloud is honest-but-curious; and (vii) network channels between layers are subject to eavesdropping. SGX hardware prevents the fog-node host OS from accessing enclave memory. Inside each fog node’s attested enclave, the sealed copy of k_{fogI} is used for AES decryption and Paillier conversion; plaintext measurements never leave the enclave. sk_P , k_{store} , and decrypted aggregates are visible only inside the dedicated storage-preparation enclave provisioned by the KMM. The model permits partial loss of the current aggregation window if a fog node fails before its aggregate is delivered to the KMM.

1) *Integrity guarantee*: Because the sensor and storage stages use AES-128-GCM, accepted payloads carry integrity protection under the standard nonce-uniqueness assumption. Each protected payload is produced and verified as

$$(C, T) = Enc_{GCM}(k, n, m, A) \quad (2)$$

$$m \text{ or } \perp = DecVer_{GCM}(k, n, C, A, T) \quad (3)$$

where T is an authentication tag, n is a nonce, and A is associated data. This provides payload integrity for all accepted ciphertexts; the edge gateway rejects any reading whose tag fails verification.

The model does not address denial-of-service attacks on the physical infrastructure.

IV. OUR PROPOSED SCHEME

The proposed scheme is organized into seven phases. The notations used in this paper are defined in Table ??.

TABLE I
SUMMARY OF NOTATION

Notation	Description
System Parameters	
λ	Security parameter
τ	Fog node workload threshold
W	Aggregation window duration, set to 500 ms
T_g	Gossip broadcast interval, set to 500 ms
T_{ack}	Sensor ACK timeout for direct failure detection, set to 100 ms
T_{KMM}	KMM attested provisioning latency, approximately 50 ms
m_i	Sensor measurement at time index i
$scale$	Integer scaling factor for Paillier encoding
f	Aggregation function supported by Paillier, e.g., sum, mean, weighted sum
p	CP-ABE access policy
$slot_map$	KMM mapping from sensor identifier to assigned fog node
$LastKnownWorkload(F_i)$	Most recent workload value for F_i from the gossip table
$LastKnownCapacityScore(F_i)$	Most recent CapacityScore for F_i cached from the gossip table, computed using the LS sensor-reading weight configuration
k_{gossip}	Gossip fan-out (number of neighbors contacted per round), set to 3
$miss_gossip(F_i)$	Number of consecutive gossip rounds in which F_i has not responded
Key Material	
k_{out}	Fog-scoped AES key assigned to fog node F_i
pk_P, sk_P	Paillier public and secret key pair
pk_{ABE}, mk_{ABE}	CP-ABE public and master secret keys
k_{store}	AES symmetric key for cloud-storage encryption
sk_{ABE}	CP-ABE secret key held by authorized Kubernetes services
ek_A	SGX sealing key of enclave on Fog A
ek_B	SGX sealing key of enclave on Fog B
$quote_A$	Remote attestation quote of Fog A’s enclave
$quote_B$	Remote attestation quote of Fog B’s enclave
$sealed_k$	Key material sealed under an enclave hardware key
$sealed_kFogA$	Fog A scoped key sealed for its home enclave
$sealed_kFogAB$	Fog A scoped key provisioned by KMM to Enclave B for one delegation epoch
Ciphertexts	
C^{AES}	AES ciphertext of sensor measurement m_i
C^P	Paillier ciphertext of $m_i \times scale$
C^{zero}	Paillier encryption of zero for a vacated slot
C_{agg, F_i}^P	Running Paillier aggregate at fog node F_i
C_{agg}	KMM-combined aggregate across all fog nodes
C_{data}	$Enc_{AES}(k_{store}, [D_{agg}, meta_data])$: single AES-GCM object stored in cloud
C_k	CP-ABE ciphertext of k_{store}
Cryptographic Operations	
$Enc_{AES}(k, m)$	AES encryption
$Dec_{AES}(k, C)$	AES decryption
$Enc_P(pk, m)$	Paillier encryption
$Dec_P(sk, C)$	Paillier decryption inside enclave
$\oplus$	Paillier homomorphic addition
$SGX_Attest(E)$	Remote attestation for enclave E
$Seal(k, ek)$	Seal key material under enclave key ek
$Unseal(sealed_k, ek)$	Recover sealed key material inside an enclave
$Enc_{CP-ABE}(pk, m, P)$	CP-ABE encryption
$Dec_{CP-ABE}(sk, C)$	CP-ABE decryption

The notation in Table ?? is canonical and used consistently across prose, equations, and Algorithm 1.

A. System Process

The overall secure data-processing workflow of the proposed framework is illustrated in Fig. ??.

The process includes sensor-level AES-GCM encryption, edge-gateway tag verification, SGX-protected AES-to-Paillier conversion, Paillier slot aggregation, per-reading TEE delegation, KMM window combine, and secure cloud storage with CP-ABE-based access control.

Paillier partial homomorphic encryption is selected over fully homomorphic schemes such as CKKS and BFV because the target aggregation functions—sum, mean, and weighted sum—require only additive homomorphism. This design choice eliminates noise-budget management, rotation-key generation, and polynomial-degree tuning, reducing implementation complexity while maintaining IND-CPA security under the decisional composite residuosity assumption.

1) *Phase 1 Initialization*: During initialization, the KMM generates fog-scoped AES keys, Paillier keys, CP-ABE keys, and the storage key. After successful remote attestation, each fog enclave is provisioned only with its own sealed fog-scoped key.

flow_diagram.pdf

Fig. 2. Proposed Secure IIoT Data Processing Framework with TEE-Enforced Key Transformation, Paillier Slot Aggregation, and KMM Window Combine

$$\forall F_i : k_{fogI} \leftarrow KeyGen_{AES}() \quad (4)$$

$$(pk_P, sk_P) \leftarrow KeyGen_{Paillier}(2048) \quad (5)$$

$$(pk_{ABE}, msK_{ABE}) \leftarrow Setup_{CP-ABE}(\lambda) \quad (6)$$

$$k_{store} \leftarrow KeyGen_{AES}() \quad (7)$$

$$\forall F_i : (ek_i, quote_i) \leftarrow SGX_Attest(Enclave_i) \quad (8)$$

$$sealed_kFogA = Seal(k_{fogA}, ek_A) \quad (9)$$

The KMM retains sk_P and k_{store} and seals them for its own storage-preparation enclave:

$$\begin{aligned} sealed_sk_P &= Seal(sk_P, ek_{SP}), \\ sealed_k_{store} &= Seal(k_{store}, ek_{SP}) \end{aligned} \quad (10)$$

where ek_{SP} is the sealing key of the dedicated storage-preparation enclave. These keys are never provisioned to fog-node enclaves.

The KMM builds $slot_map : sensor_id \rightarrow fog_node$, stores $fog_node \rightarrow k_{fogI}$ for delegation provisioning, sets $W = T_g = 500\text{ms}$, and initializes each fog node aggregate as

$$C_{agg, F_i} = Enc_P(pk_P, 0). \quad (11)$$

2) **Phase 2 Sensor Data Encryption:** Each sensor encrypts its readings using AES-128-GCM under the fog-scoped key of its assigned fog node (four readings per window as stated in Section III-A). A sensor assigned to Fog A encrypts only with k_{fogA} . The edge gateway verifies the AES-GCM authentication tag using its provisioned copy of the fog-scoped key before forwarding the ciphertext to the fog node. A single fog-node failure therefore affects 80 of 400 readings (20% of window traffic) under an even sensor distribution.

$$(C_i^{AES}, T_i) = Enc_{GCM}(k_{fogA}, n, m_i, A) \quad (12)$$

3) **Phase 3 Per-Reading Workload Check and Domain Conversion:** For every incoming ciphertext at Fog A, delegation is decided immediately using the latest gossip value.

$$LastKnownWorkload(F_A) \geq \tau \quad (13)$$

If the condition is false, Fog A processes the reading inside its enclave and adds the Paillier ciphertext to its running aggregate in host memory.

$$m_i = Dec_{AES}(k_{fogA}, C_i^{AES}) \quad (14)$$

$$C_i^P = Enc_P(pk_P, \lfloor m_i \cdot scale \rfloor) \quad (15)$$

$$C_{agg, F_A} \leftarrow C_{agg, F_A} \oplus C_i^P \quad (16)$$

If the condition is true, Fog A zero-fills the vacated slot, notifies the KMM of the target fog node, and forwards the original AES ciphertext through the delegated processing path.

$$C_{zero} = Enc_P(pk_P, 0) \quad (17)$$

$$C_{agg, F_A} \leftarrow C_{agg, F_A} \oplus C_{zero} \quad (18)$$

4) **Phase 4 TEE Delegation:** When delegation is triggered, Fog A selects the target using the proposed CapacityScore (S6) rule over available non-primary candidates from its gossip table and notifies the KMM. Candidates with $Workload(F_j) < \tau$ are preferred, with fallback to all available non-primary nodes if every candidate is saturated. Fog A does not transfer keys to Fog B, and the sensor is not contacted.

$$F_{target} = \arg \max_{F_j \in C} CapacityScore(F_j, \tau_{type}) \quad (19)$$

$$Notify_{KMM}(s, F_A, F_{target}) \quad (20)$$

$$Verify(quote_B) = true \quad (21)$$

The KMM opens an attested channel to Enclave B, provisions Fog A's scoped key to Enclave B for this delegation epoch, and updates $slot_map : sensor\ s \rightarrow Fog\ B$.

$$sealed_kFogA@B \leftarrow Provision_{KMM \rightarrow Enclave_B}(k_{fogA}) \quad (22)$$

$$k_{fogA} \leftarrow Unseal(sealed_kFogA@B, ek_B) \quad (23)$$

Fog B decrypts the delegated reading inside its enclave and accumulates the Paillier ciphertext in its own running aggregate. Delegated keys are revoked after the window closes, bounding the delegation exposure to one epoch.

$$m_s = Dec_{AES}(k_{fogA}, C_s^{AES}) \quad (24)$$

$$C_s^P = \text{Enc}_P(pk_P, \lfloor m_s \cdot \text{scale} \rfloor) \quad (25)$$

$$C_{agg, F_B} \leftarrow C_{agg, F_B} \oplus C_s^P \quad (26)$$

$$\text{Revoke}_{KMM}(k_{fogA}, \text{Enclave}_B) \text{ after } W \quad (27)$$

5) Phase 4b ACK-Based Fault Detection with Gossip Redirect: Gossip-only fault detection has a detection gap of $rT_g = 3 \times 500 \text{ ms} = 1.5 \text{ s}$. During this interval, up to three aggregation windows may be affected before neighboring fog nodes identify the failed node. The proposed ACK-based mechanism reduces this gap to approximately $T_{ack} + T_{KMM} = 100 \text{ ms} + 50 \text{ ms} = 150 \text{ ms}$ by allowing sensors to detect fog-node silence directly, notify the KMM, and redirect the encrypted reading using their local gossip table and the proposed CapacityScore (S6) LS-default selector after delegated key provisioning.

a) Operation: Each time a sensor sends C_i^{AES} to its assigned fog node, it starts a short ACK timer. The fog node returns a lightweight acknowledgement immediately after receiving and queuing the reading. If the ACK does not arrive within T_{ack} , the sensor performs the following local redirect procedure:

- The sensor consults its local gossip status table, which is maintained using $k \times 28$ bytes of RAM.
- It selects $F_{backup} = \arg \max_{F_j} \text{CapacityScore}(F_j, \text{LS})$ while excluding the silent fog node. Because raw sensor ciphertexts carry no task-type metadata, the LS weight configuration is used by default, as sensor readings must arrive within the active 500 ms aggregation window, satisfying the latency-sensitive routing criterion.
- It sends a small redirect notification to the KMM indicating that Fog A is silent and Fog B has been selected.
- The KMM provisions $\text{sealed}_k \text{FogA@B}$ to Enclave B through an attested channel, with expected provisioning latency of approximately 50 ms.
- It retransmits C_i^{AES} directly to F_{backup} .
- Enclave B decrypts with the provisioned k_{fogA} , so the blast radius is limited to Fog A's sensor group and does not expose keys for other fog nodes.

b) Sensor gossip table memory cost: Each fog-node status entry in the sensor's local table requires 28 bytes:

TABLE II
SENSOR-SIDE GOSSIP STATUS ENTRY

Field	Size
fog_node_id	4 bytes
workload	4 bytes
latency	4 bytes
queue_length	4 bytes
timestamp	8 bytes
capacity_score_ls	4 bytes
Total per fog node	28 bytes

For $k = 10$ fog nodes, the table consumes $10 \times 28 = 280$ bytes. On a Cortex-M3 class IIoT sensor with 32 KB RAM, this is less than 1% of available RAM and remains negligible

for the proposed deployment model. No hardware upgrade is required.

c) Sensor gossip table dissemination: The edge gateway rebroadcasts a signed summary of the current fog gossip table to all downstream sensors every $T_g = 500 \text{ ms}$. Each 280-byte summary is HMAC-authenticated with the receiving sensor's fog-scoped key k_{fogI} , allowing the sensor to verify that the table is fresh and untampered. This dissemination adds one 280-byte broadcast per T_g interval per sensor, corresponding to approximately 0.56 KB/s at $T_g = 500 \text{ ms}$ —a negligible overhead on 20 Mbps or higher links. This cost is included in the $1.055\times$ message-overhead figure reported in E6.

d) Three-layer fault handling hierarchy: The complete fault-handling system uses three layers, each covering failures that the previous layer cannot handle:

e) Interaction with the Paillier slot protocol: When Layer 2 triggers mid-window, the slot protocol handles redirected readings similarly to a normal delegation event after the ACK timeout. If Fog A fails, its ACKs stop arriving at assigned sensors. Those sensors detect the silence within $T_{ack} = 100 \text{ ms}$ and retransmit subsequent C_i^{AES} values to Fog B. However, because Fog A is dead, it cannot zero-fill its own vacated slots in the current window. The KMM therefore handles the edge case at window close:

- The KMM detects that Fog A did not submit C_{agg, F_A} at window close.
- Fog A's contribution is marked as zero for that window.
- The KMM combines the remaining fog-node aggregates as $C_{agg}^{final} = C_{agg, F_B} \oplus C_{agg, F_C} \oplus \dots$.
- Readings already processed by Fog A before failure are lost for that partial window if Fog A never submitted its aggregate.
- Readings after the T_{ack} redirect are captured at Fog B.

The net result is partial-window recovery. Readings processed by Fog A before failure are lost if Fog A never submits its aggregate, while readings after ACK-timeout redirect are captured at Fog B. Data loss is bounded to approximately the ACK redirect path rather than the full 1.5 s gossip fallback interval.

6) Phase 4c Window Close and KMM Combine: At every $W = 500 \text{ ms}$ window close, each fog node sends one aggregate ciphertext to the KMM. The KMM performs only homomorphic addition and does not decrypt.

$$C_{agg}^{final} = C_{agg, F_1} \oplus C_{agg, F_2} \oplus \dots \oplus C_{agg, F_k} \quad (28)$$

The KMM then sends C_{agg}^{final} to an attested enclave for storage preparation and each fog node resets its aggregate to $\text{Enc}_P(pk_P, 0)$ for the next window.

7) Phase 5 Secure Storage Preparation: The KMM forwards C_{agg}^{final} to the dedicated storage-preparation enclave, the sole holder of sk_P and k_{store} . Inside this enclave, the Paillier aggregate is decrypted and re-encrypted for compact cloud storage; both keys and the aggregate plaintext never leave enclave memory.

$$D_{agg} = \text{Dec}_P(sk_P, C_{agg}^{final}) \quad (29)$$

TABLE III
THREE-LAYER FAULT HANDLING HIERARCHY

Layer	Trigger	Who acts	Detection time	Data loss	Mechanism
1	Fog self-detects $Workload \geq \tau$	Fog node	Immediate per reading	Zero	KMM-provisioned TEE delegation; sensor unaware
2	No ACK within T_{ack}	Sensor and KMM	$T_{ack} + T_{KMM} \approx 150\text{ ms}$	$\approx 150\text{ ms}$	Sensor notifies KMM, KMM provisions k_{fogA} to Enclave B, sensor retransmits C_i^{AES}
3	$r = 3$ missed gossip rounds	Fog neighbors and KMM	$rT_g = 1.5\text{ s}$	$\approx 1.5\text{ s}$ fallback	Neighbor detection and KMM reassignment

$$C_{data} = Enc_{AES}(k_{store}, \{D_{agg}, \text{metadata}\}) \quad (30)$$

$$C_k = Enc_{CP-ABE}(pk_{ABE}, k_{store}, P) \quad (31)$$

8) **Phase 6 Cloud Storage:** The cloud stores only the AES-encrypted aggregate and the CP-ABE protected storage key.

$$Cloud \leftarrow \{C_{data}, C_k\} \quad (32)$$

9) **Phase 7 Authorized Access:** Authorized analytics services recover the storage AES key through CP-ABE and decrypt the aggregate.

$$k_{store} = Dec_{CP-ABE}(sk_{ABE}^{K8s}, C_k) \quad (33)$$

$$\{D_{agg}, \text{metadata}\} = Dec_{AES}(k_{store}, C_{data}) \quad (34)$$

10) **Phase 6b Trust Model: Fog-Scoped Key Isolation:** The original single-global-key design creates a broad compromise domain because one broken fog enclave can expose every sensor stream. The proposed design replaces that model with fog-scoped keys, so each fog node holds only the AES key for its assigned sensor group. The KMM remains the trust anchor and provisions delegated keys only to attested enclaves for bounded delegation epochs.

The Key Management Module is the system trust anchor. It is assumed to be deployed in a hardened, high-availability environment separate from fog infrastructure. Compromise of the KMM exposes all fog-scoped keys. Compromise of a single fog node's SGX enclave, which requires physical hardware attack beyond the standard honest-but-curious fog-host threat model, exposes only that node's assigned sensor group's decryption key for the current delegation epoch. Historical data already stored as AES ciphertexts in the cloud remains protected by k_{store} and CP-ABE access control.

After each delegation window closes, the KMM revokes k_{fogA} from Enclave B. Enclave B holds the delegated key only for the duration of one 500 ms window, limiting the exposure period even if Enclave B is compromised later.

The load-balancing and fault-tolerance mechanisms share the same per-reading KMM-provisioned reassignment infrastructure. Load balancing is triggered when $LastKnownWorkload(F_i) \geq \tau$ for an incoming reading. Fault tolerance is triggered either by sensor ACK timeout or by r consecutive missed gossip rounds; in both cases, the KMM provisions only the failed or overloaded fog node's scoped key to the receiving enclave. The current failure window may contain a partial aggregate if the failed node did not deliver C_{agg, F_i} to the KMM before failure.

11) **Paillier Slot Mechanics and Storage Reduction:** The slot protocol preserves aggregate correctness when delegation occurs mid-window. Under normal operation, Fog A accumulates all readings assigned to it as $C_{agg, F_A} = C_1^P \oplus \dots \oplus C_n^P$. If sensor s is delegated during the same window, Fog A adds $C_{zero} = Enc_P(pk_P, 0)$ for the vacated slot while Fog B adds the delegated reading C_s^P to C_{agg, F_B} . At window close, KMM computes $C_{agg}^{final} = C_{agg, F_A} \oplus C_{agg, F_B} \oplus \dots$, producing the same sum as if no delegation had occurred.

Paillier slot aggregation reduces the number of ciphertexts stored per window from n sensor ciphertexts to one aggregate ciphertext. For $n = 100$ sensors, this changes the per-window storage pattern from 100 individual readings to a single Paillier aggregate before compact AES storage preparation. The supported aggregation functions are sum, mean, and weighted sum; variance, standard deviation, threshold count, maximum, and minimum require multiplication or comparison and are outside the Paillier-only scope.

B. Load Balancing Mechanism

1) **Workload Monitoring and Overload Detection:** Each fog node continuously monitors its system resources to determine its processing capability. Before aggregation, each raw metric is normalized to the range $[0, 1]$ as

$$\begin{aligned} \widehat{CPU}_i &= \min\left(\frac{CPU_i^{used}}{CPU_i^{cap}}, 1\right), \\ \widehat{RAM}_i &= \min\left(\frac{RAM_i^{used}}{RAM_i^{cap}}, 1\right), \\ \widehat{BW}_i &= \min\left(\frac{BW_i^{used}}{BW_i^{cap}}, 1\right) \end{aligned} \quad (35)$$

The workload of fog node F_i is then computed as

$$Workload(F_i) = 0.5 \widehat{CPU}_i + 0.3 \widehat{RAM}_i + 0.2 \widehat{BW}_i \quad (36)$$

where CPU_i^{used} , RAM_i^{used} , and BW_i^{used} denote current usage and CPU_i^{cap} , RAM_i^{cap} , and BW_i^{cap} denote node capacity for each resource. The weights 0.5, 0.3, and 0.2 reflect the relative importance of each metric in fog processing environments, where CPU availability is typically the dominant bottleneck, followed by memory and network bandwidth respectively.

In this framework, $\tau = 0.8$, indicating that the node is operating at 80% or higher *normalized* composite resource utilization. This value balances responsiveness and stability: a threshold below 0.7 risks unnecessary delegation during transient load spikes, while a threshold above 0.85 leaves insufficient headroom for the additional cryptographic operations required during secure handoff.

$$Workload(F_i) \geq \tau \quad (37)$$

TABLE IV
COMPROMISE BLAST RADIUS ACROSS KEY-PROVISIONING DESIGNS

Compromise scenario	Global AES key	All-at-init keys	Fog-scoped k_{fogI}	Reason
Fog host OS reads memory	None	None	None	SGX blocks host access to enclave memory
Fog SGX enclave physically broken	All sensors	All sensors	Only that fog node's sensor group	Fog-scoped keys bound the exposed decryption key
KMM compromised	All sensors	All sensors	All fog groups	KMM is the system trust anchor
Network eavesdropping	None	None	None	Transmissions remain AES-, Paillier-, or attested-channel protected
Fog B compromised during delegation	All sensors	All sensors	Fog A and Fog B groups only	KMM provisions only k_{fogA} to Enclave B

To maintain awareness of the system state, fog nodes exchange status information through a gossip-based communication protocol. At every gossip interval $T_g = 500\text{ ms}$, each fog node randomly selects $k = 3$ neighboring nodes and sends a lightweight status message containing its current workload, queue length, network latency estimate, current aggregation window, and maximum delay tolerance.

Upon receiving a gossip message, the neighbor updates its local status table and may further propagate the information to other nodes in subsequent gossip rounds. Through this decentralized information exchange, fog nodes gradually build and maintain a *LastKnownWorkload* table containing the most recent system state of nearby nodes. Gossip updates this table every T_g , while delegation reads the table on every incoming sensor reading; delegation therefore does not wait for either a gossip round or a window boundary.

2) *Load Redistribution and Fog Node Selection*: When $LastKnownWorkload(F_i) \geq \tau$ for an incoming reading, the fog node initiates a load redistribution procedure for that reading. Using the status information obtained from the gossip protocol, the overloaded node evaluates candidate fog nodes and selects the most suitable destination for task delegation. The selection is based on a *CapacityScore* (S6), which jointly reflects a node's current load, network latency, queue occupancy, and structural compatibility with the incoming task type.

The *CapacityScore* of candidate fog node F_j for a task of type τ_{type} is computed as

$$\begin{aligned} CapacityScore(F_j, \tau_{type}) = & w_1(1 - Workload(F_j)) \\ & + w_2(1 - \ell_j) \\ & + w_3(1 - q_j) \\ & + w_4 \cdot compat(F_j, \tau_{type}) \end{aligned} \quad (38)$$

where $Workload(F_j)$ is the normalized workload from Eq. (??). For dimensional consistency, latency and queue occupancy are normalized to $[0, 1]$ as

$$\begin{aligned} \ell_j &= \frac{lat_j}{40.0}, \\ q_j &= \min\left(\frac{\sum_{\tau \in R_j} d_{\tau,cpu}}{CPU_j^{cap}}, 1\right) \end{aligned} \quad (39)$$

where lat_j is the node latency in milliseconds, 40.0 ms is the E5 latency-normalization bound, R_j is the set of running or queued tasks on node F_j , and $d_{\tau,cpu}$ is the CPU-core demand of task τ . The compatibility term $compat(F_j, \tau_{type}) \in [0, 1]$

captures the structural suitability of a node for the task type. For Latency-Sensitive (LS) tasks, which are bandwidth-dominant, it weights available bandwidth heavily:

$$\begin{aligned} compat_{LS}(F_j) = & 0.10(1 - \widehat{CPU}_j) \\ & + 0.10(1 - \widehat{RAM}_j) \\ & + 0.80(1 - \widehat{BW}_j) \end{aligned} \quad (40)$$

For Heavy Processing (HP) tasks, which require fast CPU cores, a node-speed factor ϕ_j^2 penalises structurally slow nodes regardless of current load:

$$\begin{aligned} compat_{HP}(F_j) = & 0.60 \phi_j^2(1 - \widehat{CPU}_j) \\ & + 0.30(1 - \widehat{RAM}_j) \\ & + 0.10(1 - \widehat{BW}_j) \end{aligned} \quad (41)$$

The node-speed factor ϕ_j reflects the relative processing speed of each fog node, and execution time is modeled as $T_{exec} = base_time/\phi_j$. In the evaluation, the speed factors are set to $\phi_{F_1} = 1.00$, $\phi_{F_2} = 0.50$, $\phi_{F_3} = 0.25$, $\phi_{F_4} = 0.50$, and $\phi_{F_5} = 1.00$. The ϕ_j^2 multiplier reduces the maximum HP CPU-compatibility contribution of the slowest node F_3 to $0.60(0.25)^2 = 0.0375$, compared with 0.60 for F_1 and F_5 ; the full maximum $compat_{HP}$ is therefore approximately 0.438 for F_3 and 1.0 for F_1 or F_5 .

The parameters w_1 , w_2 , w_3 , and w_4 are dynamic weights that adapt based on the incoming task type, where

$$w_1 + w_2 + w_3 + w_4 = 1 \quad (42)$$

All score components are dimensionless in $[0, 1]$ before inversion, so the *CapacityScore* remains comparable across heterogeneous fog nodes. The fog node with the highest *CapacityScore* is selected as the delegation destination:

$$F_{target} = \arg \max_{F_j \in N_i} CapacityScore(F_j, \tau_{type}) \quad (43)$$

where N_i represents the set of neighboring fog nodes discovered through the gossip-based status exchange. In the E5 implementation, N_i excludes failed nodes and the primary node. Strategies S3–S6 first restrict N_i to nodes with $Workload(F_j) < \tau$ and fall back to all available non-primary nodes only if no under-threshold candidate exists. Strategies S1 and S2 intentionally do not apply this workload filter, modelling blind random and round-robin routing under stress.

The compared E5 strategy scores are:

Algorithm 1 Runtime TEE-Assisted Paillier Slot Aggregation with Load-Balanced Delegation

Require: Initialized fog nodes $\{F_1, \dots, F_k\}$, fog-scoped keys k_{fogI} , Paillier public key pk_P , aggregation window W , workload threshold τ , and initialized aggregates $C_{agg, F_i} = Enc_P(pk_P, 0)$

Require: Latest gossip tables containing $LastKnownWorkload$, $LastKnownQueue$, $LastKnownLatency$, and $LastKnownCapacityScore$

Ensure: Updated per-fog Paillier aggregates $\{C_{agg, F_1}, \dots, C_{agg, F_k}\}$ for the current window

- 1: **for** each sensor reading C_i^{AES} assigned to fog node F_A during window W **do**
- 2: **if** F_A is unavailable **or** $LastKnownWorkload(F_A) \geq \tau$ **then**
- 3: Classify $task_type \in \{LS, HP\}$ from task metadata
- 4: Build candidate set:

$$\mathcal{C} = \{F_j : F_j \neq F_A, F_j \text{ available, } LastKnownWorkload(F_j) < \tau\}$$
- 5: **if** $\mathcal{C} = \emptyset$ **then**
- 6: $\mathcal{C} = \{F_j : F_j \neq F_A, F_j \text{ available}\}$ {Fallback when all candidates are saturated}
- 7: **end if**
- 8: Select target fog node:

$$F_{target} = \arg \max_{F_j \in \mathcal{C}} CapacityScore(F_j, task_type)$$
- 9: **if** F_A is still alive **then**
- 10: $C_{zero} = Enc_P(pk_P, 0)$
- 11: $C_{agg, F_A} \leftarrow C_{agg, F_A} \oplus C_{zero}$ {Preserve vacated slot correctness}
- 12: **end if**
- 13: Notify KMM that sensor s is delegated from F_A to F_{target}
- 14: Forward C_i^{AES} to F_{target}
- 15: KMM verifies $quote_{F_{target}}$
- 16: KMM provisions $sealed_kFogA@F_{target}$ to $Enclave_{F_{target}}$
- 17: KMM updates $slot_map : s \rightarrow F_{target}$
- 18: $k_{fogA} = Unseal(sealed_kFogA@F_{target}, ek_{F_{target}})$ inside $Enclave_{F_{target}}$
- 19: $m_i = Dec_{AES}(k_{fogA}, C_i^{AES})$ inside $Enclave_{F_{target}}$
- 20: $C_i^P = Enc_P(pk_P, \lfloor m_i \cdot scale \rfloor)$
- 21: $C_{agg, F_{target}} \leftarrow C_{agg, F_{target}} \oplus C_i^P$
- 22: **else**
- 23: $m_i = Dec_{AES}(k_{fogA}, C_i^{AES})$ inside $Enclave_A$
- 24: $C_i^P = Enc_P(pk_P, \lfloor m_i \cdot scale \rfloor)$
- 25: $C_{agg, F_A} \leftarrow C_{agg, F_A} \oplus C_i^P$
- 26: **end if**
- 27: **end for**

Algorithm 2 Fault Recovery, Window Combine, and Secure Storage

Require: Gossip table, T_g, r, T_{ack}, τ , aggregates $\{C_{agg, F_i}\}$, $sk_P, k_{store}, pk_{ABE}$, policy P

Ensure: Cloud stores $\{C_{data}, C_k\}$

- 1: **Each** fog node **gossips** $\{workload, queue, latency, \phi, timestamp\}$ every T_g
- 2: **Update** $LastKnownWorkload$, $LastKnownQueue$, $LastKnownLatency$, and $LastKnownCapacityScore$
- 3: **if** $miss_{gossip}(F_A) \geq r$ **then**
- 4: $\mathcal{C} \leftarrow \{F_j : F_j \neq F_A, F_j \text{ available, } LastKnownWorkload(F_j) < \tau\}$
- 5: **if** $\mathcal{C} = \emptyset$ **then**
- 6: $\mathcal{C} \leftarrow \{F_j : F_j \neq F_A, F_j \text{ available}\}$
- 7: **end if**
- 8: $F_{target} \leftarrow \arg \max_{F_j \in \mathcal{C}} LastKnownCapacityScore_{LS}(F_j)$
- 9: Mark F_A unavailable and redirect its remaining sensors to F_{target}
- 10: KMM provisions $sealed_kFogA@F_{target}$ to $Enclave_{F_{target}}$
- 11: KMM updates affected $slot_map$ entries
- 12: **end if**
- 13: **if** sensor s misses ACK from F_A within T_{ack} **then**
- 14: $F_{backup} \leftarrow \arg \max_{F_j \neq F_A} LocalGossipCapScore(F_j)$
- 15: Sensor notifies KMM and retransmits C_i^{AES} to F_{backup}
- 16: KMM provisions $sealed_kFogA@F_{backup}$ to $Enclave_{F_{backup}}$
- 17: $Enclave_{F_{backup}}$ decrypts C_i^{AES} , converts it to C_i^P , and updates $C_{agg, F_{backup}}$
- 18: **end if**
- 19: At window close, each available fog node sends C_{agg, F_i} to KMM
- 20: Missing fog-node aggregates are treated as zero for the current window
- 21: KMM computes $C_{agg}^{final} \leftarrow C_{agg, F_1} \oplus \dots \oplus C_{agg, F_k}$
- 22: Reset each $C_{agg, F_i} \leftarrow Enc_P(pk_P, 0)$
- 23: Revoke delegated fog-scoped keys from receiving enclaves
- 24: $D_{agg} \leftarrow Dec_P(sk_P, C_{agg}^{final})$ inside storage-preparation enclave
- 25: $C_{data} \leftarrow Enc_{AES}(k_{store}, \{D_{agg}, metadata\})$
- 26: $C_k \leftarrow Enc_{CP-ABE}(pk_{ABE}, k_{store}, P)$
- 27: Store $\{C_{data}, C_k\}$ in cloud

$$\begin{aligned} score_{S4}(F_j) &= 0.50(1 - Workload(F_j)) \\ &\quad + 0.15(1 - \ell_j) \\ &\quad + 0.35(1 - q_j) \end{aligned} \quad (44)$$

$$\begin{aligned} score_{S5}(F_j, \tau_{type}) &= w_1(1 - Workload(F_j)) \\ &\quad + w_2(1 - \ell_j) \\ &\quad + w_3(1 - q_j) \end{aligned} \quad (45)$$

$$score_{S6}(F_j, \tau_{type}) = CapacityScore(F_j, \tau_{type}) \quad (46)$$

S1 chooses uniformly at random from candidates, S2 uses a persistent round-robin index, S3 chooses the candidate with minimum $Workload(F_j)$, S4 uses fixed CapScore weights, S5 uses task-adaptive weights without a compatibility term, and S6 is the proposed CapacityScore with the compatibility term.

3) *Task-Aware Dynamic Weight Assignment*: Rather than applying fixed weights uniformly, the CapacityScore adapts based on the incoming task type. Each task submitted by an IoT device carries a *max_delay* metadata field that classifies the task into a routing-priority tier, selecting the weight configuration used in the CapacityScore formula. This field is a classification hint, not an end-to-end deadline on the fixed $W = 500ms$ aggregation window. The fog node uses this value to classify the task into one of two types.

TABLE V
TASK CLASSIFICATION BASED ON DELAY REQUIREMENT

Task Type	Condition	Example Applications
Latency-Sensitive	$max_delay \leq 100ms$	Real-time health monitoring, industrial control systems
Heavy Processing	$max_delay > 100ms$	Video frame analysis, compression tasks

Once the task type is determined, the fog node selects the corresponding weight configuration for the CapacityScore computation:

$$(w_1, w_2, w_3, w_4) = \begin{cases} (0.10, 0.35, 0.10, 0.45), & \text{Latency-Sensitive (LS)} \\ (0.15, 0.30, 0.10, 0.45), & \text{Heavy Processing (HP)} \end{cases} \quad (47)$$

S5 uses the same first three score components but omits the compatibility term. Its task-adaptive weights are $(w_1, w_2, w_3) = (0.15, 0.45, 0.15)$ for LS and $(0.40, 0.10, 0.30)$ for HP. S4 uses fixed weights $(w_1, w_2, w_3) = (0.50, 0.15, 0.35)$ for all task types.

In both S6 configurations, the compatibility weight $w_4 = 0.45$ is the dominant term, ensuring that structural node suitability drives the selection decision beyond raw load conditions. For LS tasks, $w_2 = 0.35$ gives secondary priority to network latency, since these tasks are delay-critical. For HP tasks, $w_2 = 0.30$ and the ϕ_j^2 factor inside $compat_{HP}$ applies a strong structural penalty to slow nodes, steering compute-heavy tasks away from nodes that cannot meet their execution deadlines. S4 and S6 weights sum to 1.0; S5 is deliberately a three-term adaptive ranking score with no compatibility term.

Raw sensor ciphertexts routed during fault recovery (Layers 2 and 3) use the LS configuration by default, as explained in Section ??.

The aggregation window is also fixed at $W = 500ms$, matching the gossip interval T_g but remaining operationally independent from it. Gossip periodically refreshes *LastKnownWorkload*, while the delegation check reads the current table value whenever a sensor ciphertext arrives. Therefore, a reading can be delegated at any time within $t \in [0, W]$ rather than waiting for the next window close.

C. Fault Tolerance Mechanism

Fault tolerance is essential for maintaining the reliability and availability of fog-based systems. Due to the distributed and decentralized nature of fog infrastructures, node failures may occur as a result of hardware malfunctions, unstable network conditions, or temporary resource exhaustion. Such failures can interrupt data processing tasks and degrade the performance of Industrial Internet of Things (IIoT) applications. Therefore, effective fault tolerance mechanisms are required to detect failures and ensure continuous system operation.

Failures in fog environments occur unpredictably due to the dynamic characteristics of distributed nodes. Previous studies have shown that failure events in fog-cloud infrastructures can be modeled probabilistically to estimate their likelihood over time. In particular, the occurrence of failures can be described using a Poisson distribution [?], which represents the probability of observing a certain number of failures within a given time interval. The probability of p failures occurring during the time period T can be expressed as

$$P(p) = \frac{(\mu T)^p e^{-\mu T}}{p!} \quad (48)$$

where μ denotes the average failure rate of a fog node and T represents the observation time interval. This probabilistic model reflects the stochastic nature of node failures in distributed fog environments.

From the Poisson model, the probability of observing at least one failure event in interval T is

$$P(p \geq 1) = 1 - P(0) = 1 - e^{-\mu T} \quad (49)$$

For gossip-based failure detection, the observation window is $T = rT_g$, where r is the consecutive-miss threshold and T_g is the gossip period. Substituting gives the probability that at least one failure arrives within the gossip observation window:

$$P(p \geq 1 \mid T = rT_g) = 1 - e^{-\mu r T_g} \quad (50)$$

This is a failure-arrival model, not a detector model. The deterministic worst-case detection delay of the gossip mechanism is rT_g ; smaller T_g improves detection timeliness at the cost of higher gossip traffic. The arrival probability above can be used to select T_g : if a deployment requires that any failure arrives within the observation window with probability at most β , then T_g should satisfy

$$T_g \leq \frac{-\ln(1-\beta)}{\mu r} \quad (51)$$

For $\beta = 0.95$ this gives $T_g \leq \ln(20)/(\mu r)$. In the proposed system, the gossip parameters $r = 3$ and $T_g = 500$ ms are chosen to bound the worst-case detection delay to $rT_g = 1.5$ s; the ACK-redirect path (Phase 4b) reduces effective recovery latency to 150 ms for the common single-node failure case.

Several fault tolerance techniques have been proposed in the literature, including task replication, checkpointing, and fault-tolerant scheduling. Replication-based methods create backup copies of tasks across multiple nodes so that computation can continue even when a node fails. Similarly, checkpointing techniques [?] periodically store intermediate execution states, allowing tasks to resume from the last checkpoint rather than restarting from the beginning when failures occur. While these approaches improve system reliability, they often introduce additional storage overhead and computational costs, which may not be suitable for resource-constrained fog environments.

To address these limitations, the proposed framework adopts a decentralized failure detection mechanism based on the gossip protocol. Gossip-based communication [?] was originally introduced in epidemic algorithms for distributed database maintenance, where information is propagated among nodes through random pairwise exchanges. In this approach, nodes periodically share their local state with randomly selected neighbors, allowing system information to gradually disseminate throughout the network without relying on centralized control.

In the proposed system, each fog node broadcasts a gossip message every $T_g = 500$ ms to a randomly selected subset of $k = 3$ neighboring nodes. A node is declared unavailable if it fails to respond for $r = 3$ consecutive gossip rounds, corresponding to a worst-case detection latency of $rT_g = 1.5$ s. Under the Poisson failure-arrival model, the probability that at least one failure arrives within this window is $1 - e^{-1.5\mu}$; for example, this reaches 0.95 when $\mu \geq \ln(20)/1.5 \approx 2.0$ s⁻¹, meaning detection within one gossip window is near-certain only at very high failure rates.

In the event of a network partition, a node that cannot reach any neighbor within r rounds initiates local failover by promoting the highest-scoring known neighbor as the delegation target based on the last recorded CapacityScore (using the LS-default configuration, since the partitioned node cannot determine the task type of future arrivals).

Once a node failure is detected, the system initiates recovery through either sensor-side ACK redirect or gossip-triggered reassignment. ACK redirect sends a redirect notification to the KMM, triggers provisioning of the failed fog node's scoped key to the selected backup enclave, and retransmits new encrypted readings to the backup fog node selected by CapacityScore (S6) with the LS-default weights. Gossip-triggered reassignment promotes the highest-scoring known neighbor under the same S6 LS-default rule and updates the KMM assignment state for subsequent windows. In both cases, encrypted sensor data is processed only by attested

fog enclaves, and sealed key material is never exposed to the fog host operating system. If the failed fog node did not deliver its current C_{agg, F_i} before failure, the affected window is explicitly treated as a partial aggregate; subsequent windows are processed by the promoted target node.

Overall, the integration of probabilistic failure modeling and gossip-based distributed monitoring improves the resilience of the fog-based IIoT architecture. By enabling decentralized failure detection and secure task reassignment, the proposed mechanism ensures stable system operation even when individual fog nodes become unavailable.

V. PERFORMANCE EVALUATION

A. Experimental Setup

All experiments are implemented in Python 3.13 using the `phe` 1.5.0 Paillier library with 2048-bit RSA moduli and a fixed-point scaling factor of $scale = 1000$. The simulation environment consists of five heterogeneous fog nodes F_1 – F_5 whose hardware profiles are summarized in Table ???. The nodes range from a weak single-core node (F_3 : 1 CPU core, 2 GB RAM, 20 Mbps) to strong quad-core nodes (F_1 , F_5 : 4 CPU cores, 8 GB RAM, 100 Mbps), each serving 20 sensors for a total of 100 sensors. The overload threshold is $\tau = 0.8$, the aggregation window is $W = T_g = 500$ ms, the sensor ACK timeout is $T_{ack} = 100$ ms, and the KMM provisioning latency is $T_{KMM} = 50$ ms.

TABLE VI
FOG NODE HARDWARE PROFILES USED IN SIMULATION

Node	Class	CPU Cores	RAM (GB)	BW (Mbps)	Sensors
F_1	Strong	4	8	100	20
F_2	Medium	2	4	50	20
F_3	Weak	1	2	20	20
F_4	Medium-Fast	2	4	80	20
F_5	Strong	4	8	100	20

The experiments fall into two categories. Experiments E1–E5 evaluate the prototype implementation, with E2 and E4 using a calibrated 2048-bit host-reference cryptographic benchmark for the Paillier timing path. The benchmark mode is `host_reference_2048_no_ta`: OP-TEE QEMU timing is used only as a prototype reference, not as physical hardware timing, and TA-side 2048-bit Paillier is not enabled in this QEMU prototype. Experiments E6–E8 are analytical models evaluating reading-loss rates, end-to-end deployment latency, and key-exposure counts, respectively. Where applicable, results report 95% confidence intervals; E5 uses 20 independent seeds $\{42, \dots, 61\}$, while the other simulation experiments use their experiment-specific seed counts.

B. E1: Storage Reduction via Paillier Slot Aggregation

Objective. Quantify the per-window cloud storage reduction achieved by replacing n individual sensor ciphertexts with a single Paillier aggregate followed by compact AES re-encryption.

Setup and baselines. For each sensor count $n \in \{10, 50, 100, 200, 500, 1000\}$, total cloud storage is computed

under four schemes: (i) *plaintext* ($n \times 8$ bytes), (ii) *AES-GCM per-reading* ($n \times 36$ bytes: 12-byte nonce, 8-byte ciphertext, 16-byte tag), (iii) *Paillier without aggregation* ($n \times 512$ bytes at 2048-bit key size), and (iv) *proposed* (one 512-byte Paillier aggregate, re-encrypted as a single AES-GCM object C_{data} containing D_{agg} and metadata; payload-only size is 36 bytes excluding nonce and tag). The total stored cloud object per window is C_{data} (12-byte nonce + payload + 16-byte GCM tag) plus C_k (CP-ABE ciphertext of k_{store}). Comparisons in the figure report payload-only sizes for consistency with baseline ciphertext sizes. No sensor is trusted with aggregation in any scheme.

Results. As shown in Fig. ??, the proposed method transmits and stores exactly one ciphertext per window regardless of n , reducing ciphertext count from n to 1. Against the AES-GCM per-reading baseline, byte reduction reaches $100\times$ at $n = 100$ and $1000\times$ at $n = 1000$. Against the Paillier-without-aggregation baseline, the byte reduction factor equals the ciphertext count reduction at each n , since both use 512-byte ciphertexts. After the storage preparation phase, the single Paillier aggregate is re-encrypted together with window metadata under k_{store} to produce C_{data} ; the payload component is 36 bytes (AES-GCM over the 8-byte aggregate value), with C_k stored alongside.

Key takeaway. Paillier slot aggregation reduces cloud storage from a function linear in n to a constant independent of sensor count.

C. E2: Aggregation Latency with 2048-bit Calibration

Objective. Characterize the proposed aggregation pipeline using the calibrated 2048-bit cryptographic benchmark and report the component-level contribution of fog conversion, host aggregation, and storage preparation.

Setup and baselines. For $n \in \{10, 50, 100, 200, 500\}$ sensors per window, plaintext summation and AES-GCM encrypt-then-decrypt are measured directly in Python. The Paillier stages of the proposed path use the calibrated `secure_iiot_full_crypto_bench 100 2048` mean timings: fog TA conversion 227.537 ms, host aggregate/KMM combine 1.474 ms, and storage TA preparation 215.117 ms. The benchmark uses 2048-bit Paillier ciphertexts of 512 bytes in `host_reference_2048_no_ta` mode.

Results. Fig. ?? shows the latency results. Plaintext summation and AES-GCM encryption remain negligible relative to the cryptographic stages. The calibrated proposed path requires 444.128 ms in total, composed of 227.537 ms for fog TA conversion, 1.474 ms for host aggregation/KMM combine, and 215.117 ms for storage TA preparation. This is within the 500 ms aggregation window. These values are host-reference OP-TEE QEMU benchmark values; they must not be interpreted as physical hardware timings or as TA-side 2048-bit Paillier execution.

Key takeaway. The calibrated 2048-bit reference places the proposed path within the 500 ms window, while showing that fog conversion and storage preparation dominate the latency budget.

figures/exp1_storage.pdf

figures/exp2_latency.pdf

Fig. 3. E1: Storage comparison across schemes as a function of sensor count n . The upper panel shows total bytes; the lower panel shows ciphertext count. The proposed method stores one ciphertext per window for all n .

Fig. 4. E2: Aggregation latency vs sensor count n with calibrated 2048-bit host-reference Paillier timings. The dashed line marks the 500 ms window. The right panel shows the calibrated component breakdown of the proposed method.

D. E3: Slot Protocol Arithmetic Correctness Under Delegation

Objective. Verify that the Paillier slot protocol produces numerically correct aggregates when delegation occurs mid-window, for both single-source and multi-source delegation configurations.

1) *E3a: Single-Source Delegation: Setup.* A window of $n = 100$ sensors is simulated with $k \in \{1, 5, 10, 20, 50\}$ sensors randomly delegated mid-window from primary node F_1 to backup node F_4 . For each of 100 independent trials, the true scaled integer aggregate is computed directly from plaintext readings, and the output of the full slot protocol—AES-to-Paillier enclave conversion, Paillier accumulation with zero-fill for vacated slots, KMM combine, and decryption—is compared against it.

Results. Across all values of k and all 100 trials, the slot protocol produces zero scaled-integer error. A residual quantization error of approximately 0.045 (median absolute deviation from the true floating-point sum) is observed due to fixed-point integer encoding with $scale = 1000$ and is consistent across all delegation counts. This is an inherent property of integer Paillier encoding rather than a protocol defect, and its magnitude is inversely proportional to the scaling factor.

2) *E3b: Multi-Source Delegation:* The protocol is further evaluated under simultaneous delegation from two sources (F_1 and F_2 , each with 30 sensors) to a single backup node (F_4 with 30 own sensors), with 5 sensors per source delegated to F_4 , for 100 independent trials. All 100 trials produce the correct scaled aggregate. The median quantization error is 0.041, consistent with the single-source case.

Key takeaway. The Paillier slot protocol maintains aggregate correctness under mid-window delegation for both single and multi-source configurations across all 100 trials at every delegation count tested. These results constitute an arithmetic sanity check of the protocol implementation; they do not constitute a cryptographic proof of zero-fill ciphertext indistinguishability, which is argued analytically.

E. E4: KMM Window-Combine Overhead

Objective. Characterize the latency of the KMM homomorphic window-combine operation as the number of participating fog nodes k increases from 1 to 100.

Setup. For $k \in \{1, 2, 5, 10, 20, 50, 100\}$ fog aggregates, the KMM combine stage uses the calibrated host aggregate timing from `secure_iiot_full_crypto_bench 100 2048`. The measurement is reported as a 2048-bit host-reference benchmark in `host_reference_2048_no_ta` mode. Each aggregate ciphertext is 512 bytes at the 2048-bit key size.

Results. Fig. ?? shows the calibrated host aggregate/KMM combine reference. The combine stage requires 1.474 ms and remains well within the 500 ms window budget across the tested fog-aggregate counts. The KMM input size still scales linearly with k , from 512 bytes at $k = 1$ to 51.2 KB at $k = 100$.

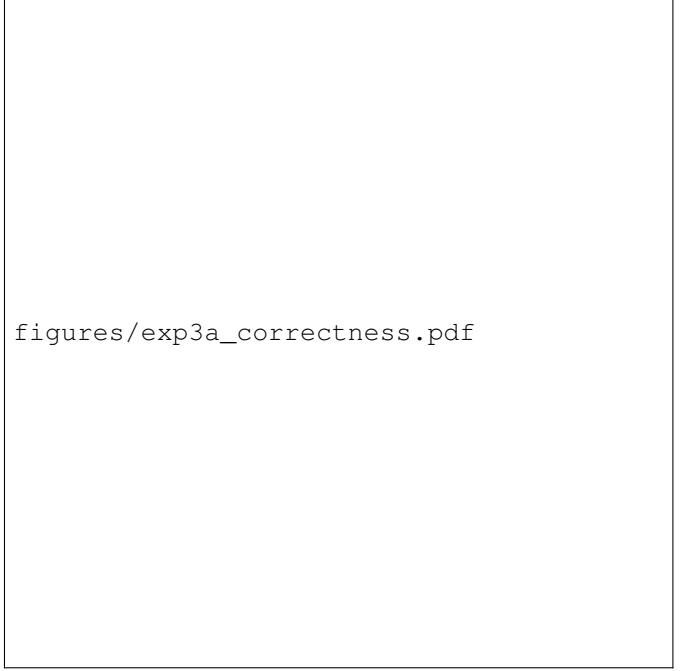


Fig. 5. E3a: Slot protocol correctness vs delegation count k (100 trials, $n = 100$ sensors). The upper panel shows 100% accuracy at all k ; the lower panel shows the quantization error due to fixed-point encoding with $scale = 1000$.

Key takeaway. The calibrated KMM combine step contributes only 1.474 ms, so it is not the limiting factor in the 500 ms aggregation window.

F. E5: Load Balancing Strategy Comparison

Objective. Compare the proposed CapacityScore strategy against five delegation baselines under heterogeneous fog capacity, bursty task arrivals, and a temporary fog-node failure.

Setup and baselines. The E5 simulator uses five fog nodes with heterogeneous CPU, RAM, bandwidth, latency, and processing-rate factor ϕ : F_1 and F_5 are high-capacity low-latency nodes, F_2 and F_4 are medium-capacity nodes, and F_3 is the slowest node. Each run spans 20 aggregation windows of 500 ms. The workload phases are normal operation (W1–5), an F_1 burst (W6–10), F_5 failure (W11–13), recovery (W14–17), and an F_4 burst (W18–20). Tasks are generated from two classes: latency-sensitive (LS) tasks with 100 ms deadlines and heavy-processing (HP) tasks with 500 ms deadlines. Each fog group has a fixed HP/LS mix, creating CPU-heavy, bandwidth-heavy, and LS-only primary identities.

Six strategies are compared: S1 random, S2 round-robin, S3 threshold/minimum-workload selection, S4 CapScore-Fixed, S5 CapScore-Adaptive, and S6 CapacityScore (proposed). S4 ranks candidates using workload, latency, and queue score with fixed weights (0.50, 0.15, 0.35, 0). S5 adapts the first three weights by task type but has no compatibility term. S6 adds a task-type compatibility term: LS routing prioritizes bandwidth availability, while HP routing applies a ϕ^2 CPU-compatibility penalty so that the slowest node F_3 is structurally disfavored.

figures/exp4_kmm_combine.pdf

Fig. 6. E4: KMM window-combine latency vs number of fog aggregates k using the calibrated 2048-bit host aggregate reference. The dashed line marks 500 ms. The right panel shows total bytes received by the KMM at each k .

for HP tasks. Results are averaged over 20 seeds $\{42, \dots, 61\}$ with 95% confidence intervals.

Metrics. The reported metrics are completion rate, deadline satisfaction rate, delegation rate, re-delegation rate, mean waiting time, workload standard deviation, resource-utilization standard deviations, per-task-type deadline miss rates, and phase-level deadline satisfaction.

Results. Aggregate results are shown in Fig. ?? and Fig. ?. All six strategies complete 100% of tasks, so the discriminating metrics are deadline satisfaction and routing quality. S6 achieves the highest overall deadline satisfaction, 78.77% ($\pm 0.88\%$), followed by S5 at 78.35% ($\pm 0.88\%$), S3 at 77.28% ($\pm 0.79\%$), S4 at 77.01% ($\pm 0.70\%$), S2 at 63.34% ($\pm 0.52\%$), and S1 at 63.10% ($\pm 0.61\%$). S6 also has the lowest mean waiting time, 101.11 ms (± 3.94 ms), compared with 103–104 ms for S3–S5 and approximately 790–854 ms for S1–S2. The proposed S6 strategy delegates fewer tasks than the other informed strategies, 43.87% ($\pm 1.21\%$) versus 47.99–49.19% for S3–S5, and no strategy produces re-delegation in this simulator configuration.

Resource-balance results show the effect of the compatibility term. S6 has the lowest bandwidth-utilization standard deviation, $\sigma_{BW} = 0.225$ (± 0.005), compared with 0.239 for S5, 0.250 for S3, 0.251 for S4, and 0.363 for S1–S2. The time-series view in Fig. ?? shows the same behavior across the burst, failure, and recovery phases. S6 does not minimize the scalar workload standard deviation because it intentionally avoids using structurally incompatible capacity; its mean workload standard deviation is 0.167, while S3–S5 are approximately 0.155–0.156. This is an expected tradeoff:

S6 accepts slightly less even scalar workload distribution to reduce deadline misses and bandwidth imbalance.

The task-type breakdown in Fig. ?? and Fig. ?? explains the deadline improvement. HP tasks miss deadlines at 15.18% under S6, compared with 17.70% for S4, 18.59% for S3, 18.67% for S5, and about 34–35% for S1–S2. S6 assigns only 1.97% of HP tasks to F_3 , while S3–S5 assign 3.32–3.77% and S1–S2 assign approximately 13.8–14.0%. Because F_3 executes HP tasks in 600 ms, exceeding the HP deadline, reducing HP placement on F_3 directly reduces deadline failures. Phase-level results in Fig. ?? show that S6 is strongest during normal operation (92.67%), recovery (93.83%), and the F_4 burst phase (84.51%). During F_5 failure, S5 performs best (48.22%) while S6 drops to 37.93%, indicating that the compatibility-heavy policy can be conservative when the candidate set is temporarily constrained by failure.

Key takeaway. E5 shows that CapacityScore improves deadline satisfaction primarily by routing HP tasks away from structurally incompatible nodes and by reducing bandwidth-utilization variance. The result is not a completion-rate improvement, since every strategy completes all tasks in this configuration; it is a deadline and routing-quality improvement under heterogeneous fog capacity and phased stress.

figures/e5_figA_delegation_quality.png

Fig. 7. E5 Figure A: delegation quality across S1–S6. Panels report completion rate, deadline satisfaction rate, delegation rate, and mean waiting time over 20 seeds with 95% CI.

G. E6: Fault Detection and Recovery Analysis

Objective. Evaluate data loss rate and recovery latency of the proposed ACK+KMM fault recovery mechanism against five representative baselines across three failure timing scenarios.

figures/e5_figB_resource_util_std.png

Fig. 8. E5 Figure B: bandwidth-utilization standard deviation across fog nodes. Lower σ_{BW} indicates more stable bandwidth distribution.

figures/e5_figD_tttotal_boxplots.png

Fig. 10. E5 Figure D: deadline miss rate by task type. S6 reduces HP misses by avoiding structurally slow HP placements.

figures/e5_figC_sigma_bw_timeseries.png

Fig. 9. E5 Figure C: σ_{BW} time series across normal, burst, failure, recovery, and second-burst phases. Shaded bands mark stress and failure intervals.

figures/e5_figE_node_scatter_w8.png

Fig. 11. E5 Figure E: HP assignment rate to F_3 , the slowest and HP-incompatible node. Lower values indicate better task-aware routing.

figures/e5_figF_phase_deadline.png

Fig. 12. E5 Figure F: phase-level deadline satisfaction over 20 seeds with 95% CI.

Setup and baselines. An analytical reading-loss model is evaluated over 30 simulation seeds and three failure scenarios: failure at the start of the observation window (*fail_0ms*), failure at 250 ms (*mid_window*), and failure at 450 ms (*late_window*). A total of 400 readings are generated per 500 ms window. The compared baselines are derived from representative fault-tolerance mechanisms reported in the literature, but they are implemented as simulator-level abstractions rather than complete reproductions of the cited systems. This distinction is important because the original studies differ in architecture, workload, and optimization objective.

Table ?? summarizes the five baselines used in E6. B1 is the direct gossip-only baseline used by the proposed architecture, where a fog failure is detected only after $r=3$ missed gossip rounds, giving $rT_g = 3 \times 500 \text{ ms} = 1500 \text{ ms}$. During this interval, readings addressed to the failed fog node are counted as lost. B2 abstracts replica-based fog fault tolerance, following the principle that task reliability can be improved by executing replicas on alternative fog resources [?]; in the simulator, each sensor reading is sent simultaneously to a primary and backup fog node, so data loss is zero under a single primary failure but message and compute units are doubled. B3 abstracts checkpoint/recovery mechanisms used in resilient cloud and service-function-chain management [?]; the simulator models periodic checkpoints at 500 ms intervals and a 500 ms restore interval, so readings after the latest checkpoint and before recovery completion may be unavailable. B4 is based on multilayer liveness detection, where multiple detector layers are combined to reduce fault detection time compared with a single heartbeat detector [?]; the simulator represents this

idea using independent sensor, fog, and KMM-layer checks and uses the earliest active detection result. B5 is based on fog/SIoT clustering fault tolerance, where clusters and standby resources support fault detection and recovery [?]; the simulator models a cluster coordinator that detects the failed fog node, selects a replacement, and reroutes subsequent readings.

TABLE VII
E6 LITERATURE-DERIVED FAULT-TOLERANCE BASELINES

Baseline	Reference role	Simulator abstraction
B1	Direct gossip-only baseline	Failure detected after $rT_g = 1500 \text{ ms}$; readings sent to the failed fog before detection are lost.
B2	Replica-based fog fault tolerance [?]	Each reading is sent to both primary and backup fog nodes; backup output is used after primary failure.
B3	Checkpoint/recovery baseline [?]	A checkpoint is written periodically; recovery resumes from the latest checkpoint after a restore delay.
B4	Multilayer liveness detection [?]	Sensor, fog, and KMM-layer detectors operate independently; detection uses the earliest active layer result.
B5	Fog-clustering fault tolerance [?]	A cluster coordinator detects the failed fog, selects a replacement, and reroutes later readings.

The proposed method uses sensor ACK timeout $T_{ack}=100 \text{ ms}$ followed by KMM key provisioning $T_{KMM}=50 \text{ ms}$, yielding a modeled recovery latency of 150 ms. The backup fog node is selected using CapacityScore (S6) with the LS weight configuration as the default, since raw sensor ciphertexts carry no task-type metadata (see Section ??). The analytical model treats backup selection as a black box: it assumes the selected node has sufficient residual capacity to accept redirected readings within the $T_{ack} + T_{KMM}$ window; the specific backup node chosen does not affect the reported loss rates or recovery latency. All results are therefore analytical estimates under a common failure-timing model and should not be interpreted as empirical end-to-end measurements of the original baseline systems.

Results. Fig. ?? shows data loss rates and recovery latency across methods. Loss rates are reported as a fraction of one 500 ms reference window (400 readings), yielding a per-window-equivalent normalization; for outages spanning multiple windows this denominator is held fixed at 400 so that all methods are compared on a common scale. With one failed fog node serving 20 sensors (80 readings per 500 ms), loss scales with the recovery-latency window. In the worst-case scenario (failure at $t=0$), the proposed method (using CapacityScore S6 with LS-default weight configuration for backup node selection) loses approximately 6% of readings per 500 ms equivalent (≈ 24 of 400 during the 150 ms recovery window). Recovery latency is measured from the first missed ACK event, not from the failure instant; worst-case failure-to-detection can be up to one inter-reading interval (125 ms) longer if the node fails immediately after a successful transmission. The proposed method compares with 60% for B1 gossip (1500 ms

outage, equivalent to three reference windows), 20% each for B3 checkpoint and B4 multilayer (500 ms detection windows), and approximately 30% for B5 clustering (750 ms window). B2 replication achieves zero data loss at the cost of doubling message overhead. The proposed method incurs a message overhead of $1.055\times$ the baseline, attributable to per-reading ACK messages and two KMM provisioning messages per recovery event. Fig. ?? illustrates the data-loss versus message-overhead tradeoff space; the proposed method is nearest to the origin among single-mechanism approaches.

Key takeaway. The proposed ACK+KMM mechanism reduces worst-case data loss from 60% (gossip-only) to approximately 6% while incurring only a 5.5% increase in message overhead, representing a favorable tradeoff relative to all compared baselines except B2 replication, which achieves zero data loss at $2\times$ message cost.

figures/exp6_data_loss.pdf

Fig. 13. E6: Data loss rate (upper) and recovery latency (lower) across six methods and three failure timing scenarios, averaged over 30 seeds with 95% CI. Results are derived from an analytical reading-loss model.

H. E7: End-to-End Pipeline Latency (Analytical Model)

Objective. Estimate end-to-end latency and per-window storage footprint of the proposed pipeline under hardware-accelerator assumptions and compare against three representative baselines across dynamic window conditions.

Setup and baselines. The model uses hardware-target latency constants: sensor AES encryption (0.001 ms, hardware AES-NI), Paillier encryption per slot (3.8 ms, hardware-assisted co-processor), Paillier homomorphic addition (0.01 ms), KMM combine inside TEE (1.0 ms), storage preparation (5.0 ms), cloud upload (10.0 ms), and TEE key delegation round-trip (150.0 ms). For $n=100$ sensors, four methods are evaluated across the same 20-window schedule

figures/exp6_tradeoff.pdf

Fig. 14. E6: Data loss vs message overhead tradeoff. Each marker represents one method; the proposed ACK+KMM method (green) is closest to the origin in loss-overhead space.

as E5: (i) *cloud-only* (raw sensor values uploaded directly, no fog processing), (ii) *fog plaintext* (fog accumulates an unencrypted sum; insecure lower bound), (iii) *Paillier-fog-convert* (per-reading Paillier encryption at the fog, no slot aggregation, one ciphertext per window sent to cloud), and (iv) *proposed*. These latency constants represent a separate target-deployment analytical model; E2 reports the calibrated 2048-bit host-reference benchmark.

Results. Fig. ?? shows pipeline latency and per-window storage. Cloud-only and fog-plaintext complete in 40 ms and 13 ms, respectively; however, cloud-only stores 100 raw values per window (800 bytes) while fog-plaintext provides no data confidentiality. The Paillier-fog-convert baseline requires 393 ms and stores one 512-byte ciphertext per window, satisfying the 500 ms budget across all 20 windows. The proposed method requires 399 ms during non-delegation windows (12 of 20 windows) and 549 ms during delegation and failure-recovery windows (8 of 20 windows), where the additional 150 ms TEE key delegation step is active. Both the proposed method and the Paillier-fog-convert baseline store one aggregate per window, compared with 100 raw items per window for cloud-only.

Key takeaway. Under hardware-target assumptions, the proposed method achieves the same aggregate storage efficiency as the Paillier-fog-convert baseline (one ciphertext per window) while additionally supporting delegation and fault recovery. The 500 ms budget is maintained in 12 of 20 windows; delegation and recovery windows incur an additional 150 ms, which is the cost of TEE key provisioning. The fog-plaintext baseline serves as a latency lower bound only

and does not satisfy the confidentiality requirements of the proposed threat model.

figures/exp7_pipeline.pdf

Fig. 15. E7: Analytical end-to-end pipeline latency (left) and per-window storage items (right) for four methods across 20 windows ($n=100$). The dashed line marks the 500 ms window budget. The proposed method exceeds this budget in 8 windows where TEE key delegation is active. **Note:** This is an analytical model using hardware-target constants; E2 reports a separate calibrated 2048-bit host-reference benchmark.

I. E8: Key Exposure Blast Radius

Objective. Quantify the number of sensors whose decryption key is exposed under four threat scenarios, comparing a global shared AES key architecture with the proposed fog-scoped key design.

Setup. The system consists of 100 sensors distributed evenly across five fog nodes (20 per node). Four compromise scenarios are evaluated using analytical exposure accounting: (i) *one fog node compromised* (F_1 enclave physically broken); (ii) *backup node compromised during active delegation* (F_4 enclave broken while holding a provisioned k_{fogA} key); (iii) *KMM compromised*; and (iv) *host OS reads enclave memory* (under the standard SGX honest-but-curious threat model). These results represent analytical exposure counts and are not the product of a cryptographic attack simulation or SGX penetration test.

Results. Fig. ?? shows the results. In scenario (i), a global key scheme exposes all 100 sensors while fog-scoped keys expose only the 20 sensors assigned to the compromised node, an 80% reduction. In scenario (ii), the backup node holds only k_{fogA} provisioned for the current delegation epoch, exposing 40 sensors (20 from F_1 and 20 from F_4 's own group) versus all 100 under a global key, a 60% reduction. In scenario (iii), KMM compromise exposes all 100 sensors under both schemes, since the KMM is the trust anchor for both; no

isolation benefit exists in this scenario. In scenario (iv), the SGX assumption prevents host OS access to enclave memory in both schemes, so no sensors are exposed under either design.

Key takeaway. Fog-scoped key isolation reduces sensor exposure by 60–80% in single-node compromise scenarios relative to a global key design. The benefit does not extend to KMM compromise, which the threat model identifies as a critical single point of failure common to both schemes.

figures/exp8_blast_radius.pdf

Fig. 16. E8: Analytical key exposure comparison between a global AES key design and the proposed fog-scoped key design across four threat scenarios (100 sensors total, 20 per node). KMM compromise exposes all sensors under both schemes.

VI. FUTURE WORK

A limitation of the present work is that the CapacityScore (S6) weights (w_1, w_2, w_3, w_4) and the compatibility term coefficients are fixed heuristics selected by task classification rather than learned from observed workload distributions. The strategy comparison in E5 demonstrates the advantage of the compatibility-aware term over simpler scoring strategies, but the specific weight values motivate data-driven adaptation. An important direction for future work is to replace the static weight table with an online learning mechanism—such as a contextual bandit or lightweight reinforcement learning policy—that adjusts weights based on observed deadline miss rates and node utilization histories across aggregation windows. Additionally, the workload update model in E5 uses a formula-based service time derived from static node profiles rather than a principled queuing model; extending the simulation to a discrete-event framework with explicit queue dynamics (e.g., M/M/c per node) would produce more accurate estimates of redelegation rates and deadline miss probabilities

at larger scales. The present evaluation covers five fog nodes, 20 windows, and 46–58 tasks per window depending on the stress phase; characterizing system behavior under hundreds of nodes and high-variance arrival rates represents a necessary step toward deployment-scale validation. Furthermore, the current Paillier-based aggregation layer supports only additive operations—sum, mean, and weighted sum. Future work will extend the scheme to CKKS-based slot batching to support multiplicative aggregation functions such as variance, product aggregation, and nonlinear statistics, and to provide approximate arithmetic over packed sensor vectors with formally bounded noise growth.

The experimental evaluation rests on several simplifying assumptions that limit direct correspondence to real deployments. The SGX trusted execution environment is entirely simulated: `sgx_enclave_process` is implemented as an ordinary Python function performing AES decryption followed by Paillier encryption, with no actual enclave boundary, no sealing key derivation, and no remote attestation exchange. The KMM provisioning latency $T_{KMM} = 50$ ms is a fixed design parameter rather than a measured quantity. Future work should implement the enclave layer using a real SGX SDK (e.g., Intel SGX SDK or Gramine) and validate provisioning latency on Skylake or Ice Lake class hardware. Similarly, the end-to-end pipeline latency in E7 is derived entirely from hard-coded hardware-target constants; these estimates should be replaced by measurements on physical fog hardware equipped with AES-NI and an SGX-capable processor. The CP-ABE cloud access control layer is fully specified in the scheme and notation but is absent from all experiment scripts: neither key generation, policy evaluation, nor attribute revocation overhead is measured. Integrating a standard CP-ABE implementation (e.g., Charm-Crypto) and characterizing its latency relative to the 500 ms aggregation window is an important open task. Finally, the multi-cluster Kubernetes analytics path described in the architecture has not been deployed or tested on any real cluster infrastructure; end-to-end validation of the authorized access path on a real Kubernetes environment would be required before the scheme can be considered production-ready.

The fault tolerance evaluation in E6 is based on an analytical reading-loss model that counts sensor readings falling within a fixed failure-to-recovery time window under three predetermined failure timings. This model does not capture cascading failures, partial node degradation, network partition effects, or the interaction between simultaneous sensor ACK timeouts and the gossip convergence process. Future work should validate the proposed ACK+KMM recovery mechanism under realistic fault injection using network emulation tools such as tc-netem or a containerized testbed, and should extend the failure model to cover correlated multi-node failures and transient network outages. The gossip protocol parameters—fan-out $k=3$, miss threshold $r=3$, and gossip interval $T_g=500$ ms—are fixed throughout the evaluation; the probabilistic detection model derived from the Poisson failure process provides a design criterion ($T_g \leq \ln(20)/(\mu r)$ for $\beta = 0.95$), but

adaptive parameter selection under varying failure rates μ has not been investigated. The KMM is currently modeled as a single trusted authority; future work may investigate threshold or multi-party key management constructions that distribute the trust anchor across a quorum of KMM instances, reducing the exposure when the KMM is compromised. Finally, the present scheme provides computational confidentiality under the IND-CPA assumption for Paillier and AES-GCM but does not address what an honest-but-curious cloud observer can infer from the sequence of decrypted aggregates over time. Investigating differential privacy mechanisms for the aggregate output, or bounding the statistical leakage of the published aggregate stream, remains an open problem for future work.